

Industrial Internship Report on "Smart City Traffic Forecasting"

**Prepared by
Dhananjay Maurya**

Executive Summary

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time.

My project was Smart City Traffic Forecasting — building a machine learning pipeline to predict hourly vehicle traffic volume at four city junctions using historical time-series data, so that traffic patterns can be anticipated ahead of time instead of only reacted to.

This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1	Preface	4
2	Introduction	6
2.1	About UniConverge Technologies Pvt Ltd	6
2.2	About upskill Campus	11
2.3	Objective	13
2.4	Reference	13
2.5	Glossary.....	13
3	Problem Statement.....	15
4	Existing and Proposed solution.....	16
5	Proposed Design/ Model	17
5.1	High Level Diagram (if applicable)	17
5.2	Low Level Diagram (if applicable)	17
5.3	Interfaces (if applicable)	18
6	Performance Test.....	19
6.1	Test Plan/ Test Cases	19
6.2	Test Procedure.....	20
6.3	Performance Outcome	20
7	My learnings.....	22
8	Future work scope	22

1 Preface

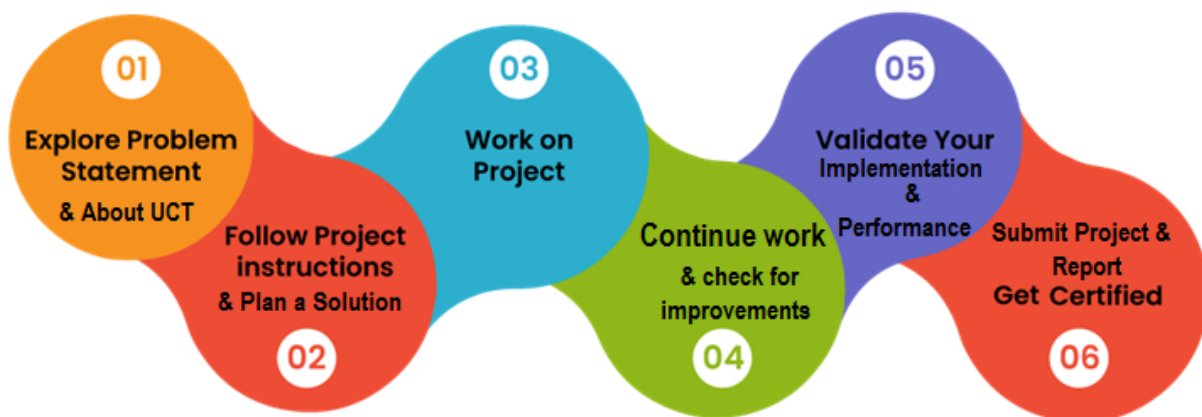
This report covers the full six weeks of my ML internship, during which I built an end-to-end traffic forecasting pipeline for Project 9 – Smart City Traffic Forecasting, from raw data to a compared set of trained models.

An internship like this matters for career development because it forces you to work with a real, messy dataset end-to-end instead of a clean textbook example — setting up the environment, exploring and cleaning data, engineering features, training multiple models, and honestly comparing their results.

My project/problem statement: forecast hourly traffic volume at four city junctions using historical DateTime/Junction/Vehicle-count data, to support smart city traffic management and congestion planning.

This opportunity was provided through upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

The program was planned week by week: Week 1 was project familiarization and environment setup; Week 2 was EDA, feature engineering and baseline models; Week 3 was SARIMA, LSTM and error analysis; Week 4 was per-junction modeling, further tuning, and a final model comparison.



My biggest learning was technical rigor around time-series data specifically — catching and fixing a data leakage bug from an initial random train/validation split, learning that a tuned model doesn't automatically beat a default one, and seeing first-hand that deep learning (LSTM) isn't automatically better than a well-tuned Random Forest on a moderately-sized dataset. Overall it was a great hands-on experience.

Thanks to my mentors at upskill Campus and UniConverge Technologies for their guidance through the weekly review process, and to my peers for useful discussions along the way.

To juniors: don't just chase the fanciest model — spend real time on EDA and get your train/validation split right first. A simple model on clean, well-understood data will beat a complex model on a leaky pipeline every time.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies** e.g. **Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoSrWAN), Java Full Stack, Python, Front end** etc.



i. UCT IoT Platform ()

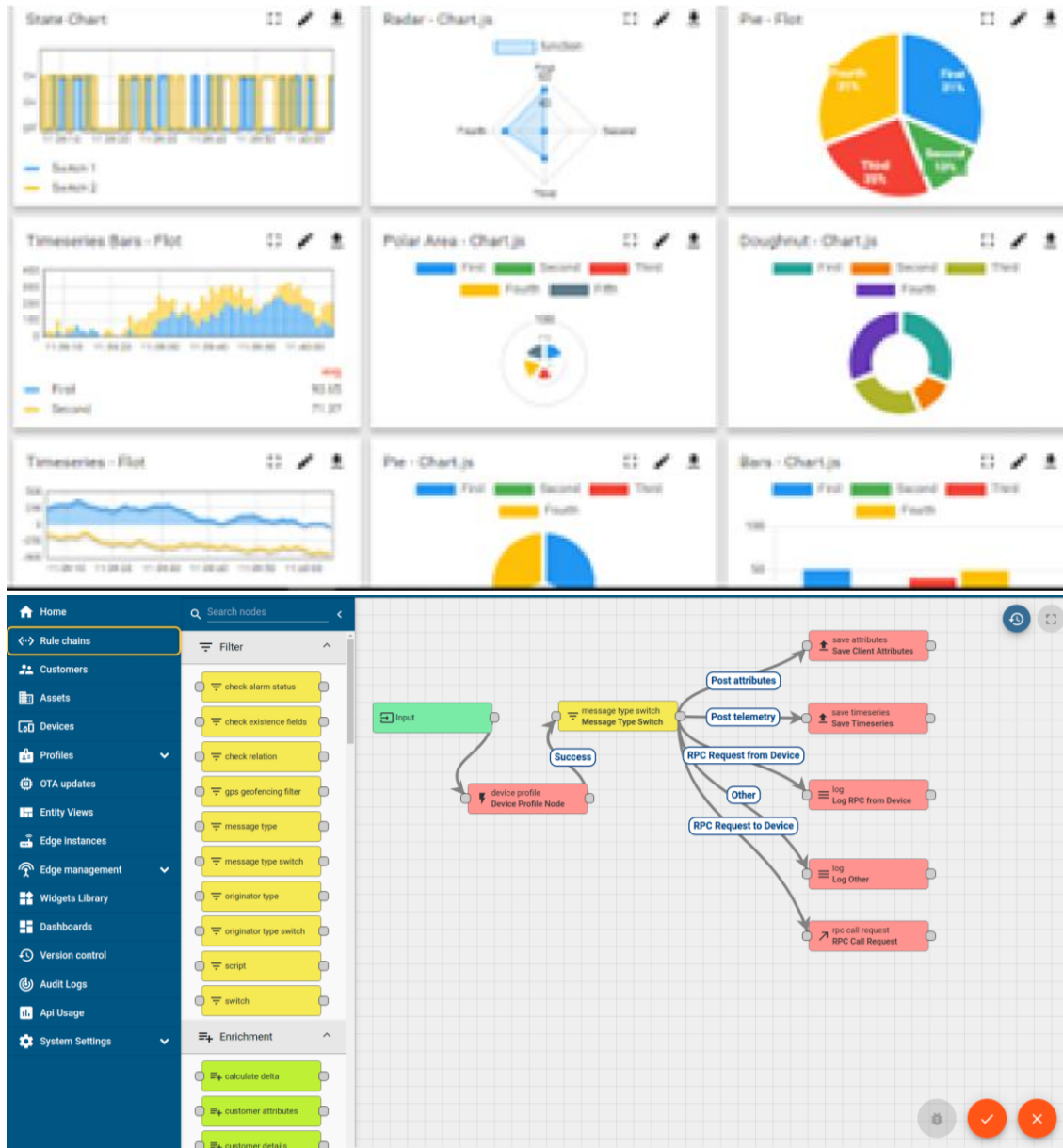
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA

- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine



FACTORY
WATCH

ii. Smart Factory Platform ()

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleash the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they want to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i



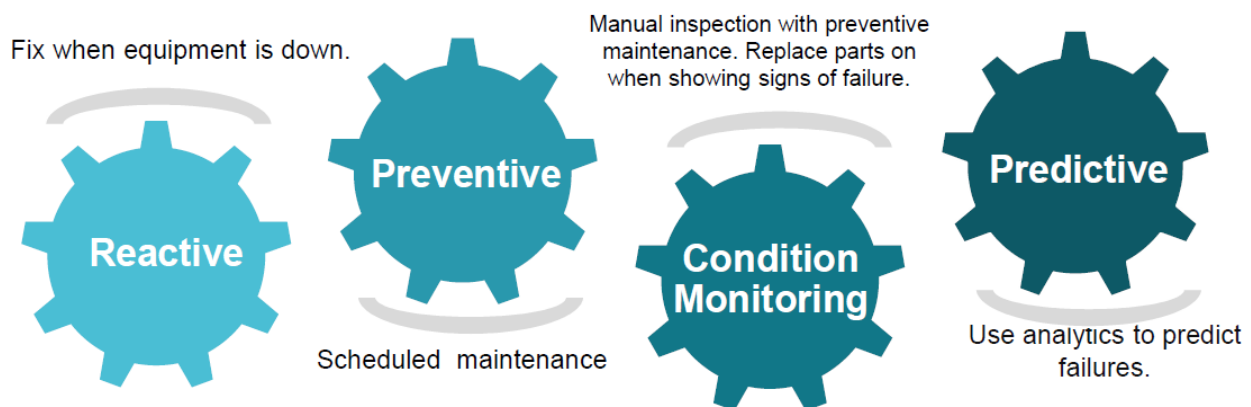


iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN technology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

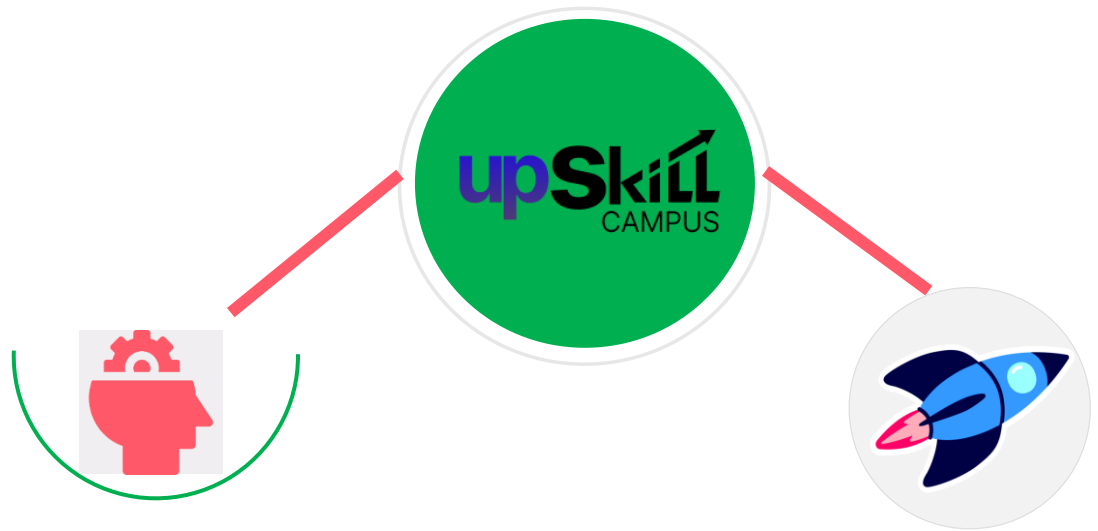
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

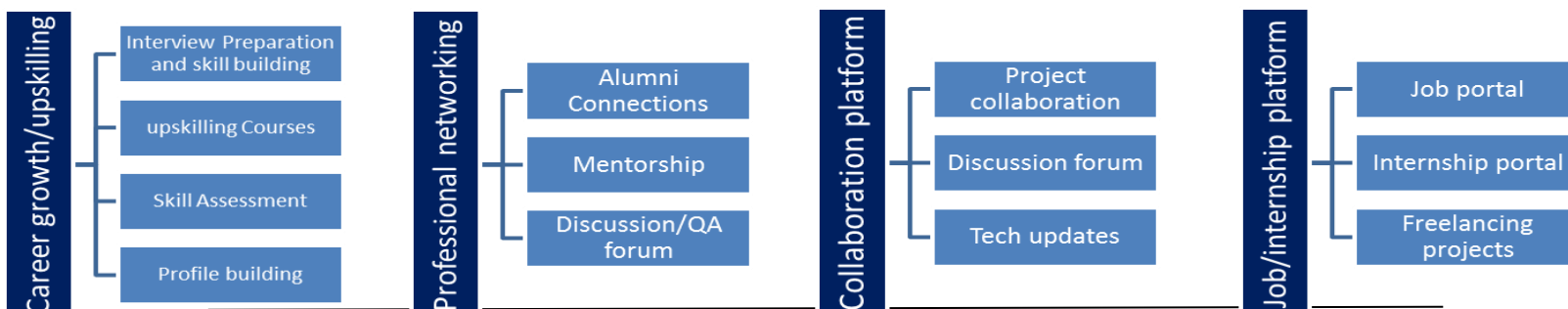
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

<https://www.upskillcampus.com/>



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- get practical experience of working in the industry.
- to solve real world problems.
- to have improved job prospects.
- to have Improved understanding of our field and its applications.
- to have Personal growth like better communication and problem solving.

2.5 Reference

- [1] Traffic Prediction Dataset (hourly vehicle counts, 4 junctions, Nov 2015 - Jun 2017).
- [2] scikit-learn, statsmodels, and TensorFlow/Keras documentation.
- [3] Weekly progress reports, Weeks 1-4 (this internship).

2.6 Glossary

Terms	Acronym
-------	---------

MAE	Mean Absolute Error
RMSE	Root Mean Squared Error
SARIMA	Seasonal AutoRegressive Integrated Moving Average
LSTM	Long Short-Term Memory (a type of recurrent neural network)
EDA	Exploratory Data Analysis

3 Problem Statement

In the assigned problem statement (Project 9), the goal was to design and build a machine learning solution that forecasts short-term traffic volume (vehicle counts) at multiple city junctions using historical, timestamped traffic-count data. The dataset spans four junctions from November 2015 to June 2017 at hourly resolution (48,120 rows total), with each row containing a DateTime, Junction ID, vehicle count, and a unique ID; Junction 4's sensor came online later than the others, so its history is noticeably shorter than Junctions 1–3, which is itself part of the problem to be handled rather than just a footnote. This matters in practice because uncontrolled, unpredicted congestion at city junctions leads to longer commute times, higher fuel consumption, increased emissions, and reduced effectiveness of emergency-response routing, and most junctions today are managed reactively with fixed-timing signals and no forward-looking estimate of expected traffic. The forecasting model therefore needs to capture overlapping daily and weekly seasonal patterns in traffic, handle junctions that differ substantially in both volume and volatility without under-serving the busiest or most erratic one, strictly separate past and future data during evaluation so that accuracy numbers are honest rather than inflated by leakage, and distinguish genuine traffic spikes from real local events out of statistical outliers instead of blindly discarding them. A successful outcome is a pipeline that predicts next-hour vehicle counts per junction with materially lower error than a naive baseline, evaluated on a strictly future, unseen time window, and benchmarked across more than one modeling approach — classical regression, tree-based, seasonal statistical, and deep learning — so the final model choice is evidence-based rather than assumed, ultimately producing forecasts accurate and reliable enough to support real smart-city interventions such as adaptive signal timing, congestion alerts, and route planning rather than remaining just an academic exercise.

4 Existing and Proposed solution

Traditional traffic management in most cities still relies on fixed-timing traffic signals and manual observation, with limited or no forward-looking prediction. Where forecasting is used, it is often a single generic model applied uniformly, without accounting for how different junctions can have very different traffic patterns and volumes, or without properly separating training/validation data in time — which can make reported accuracy numbers misleading (a mistake I ran into myself in Week 2, before fixing it).

My proposed solution is a data-driven forecasting pipeline that: (1) engineers time-aware features (hour, day-of-week, holiday flag, lag values, rolling averages) from raw timestamped counts, (2) trains and fairly compares four different modeling approaches — Linear Regression, Random Forest, SARIMA, and an LSTM — on a strict chronological train/validation split to avoid data leakage, and (3) evaluates whether junction-specific models improve on a single combined model.

The value addition is in the comparison and error analysis, not just a single trained model: the project shows which approach genuinely works best for this kind of data (Random Forest, in this case), where each model's forecasts break down (rush hours, the busiest junction), and quantifies the real trade-off of splitting models per junction — useful, evidence-based decisions a city could actually act on rather than a single black-box prediction.

4.1 Code submission (GitHub link): <https://github.com/MauryaQbit/smart-city-traffic-forecasting.git>

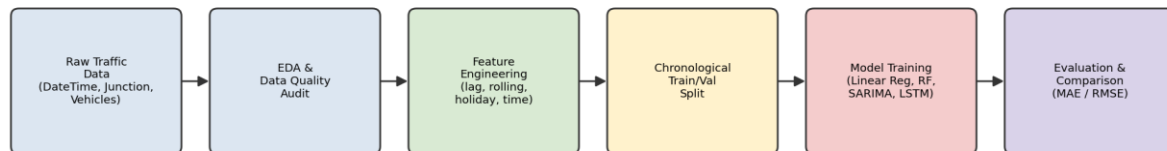
4.2 Report submission (GitHub link): <https://github.com/MauryaQbit/smart-city-traffic-forecasting/tree/main/reports>

5 Proposed Design/ Model

The solution follows a standard applied ML pipeline, adapted for time-series data: raw data goes through a data-quality audit and EDA, then feature engineering (temporal, lag, and rolling-window features, plus a holiday flag), then a chronological train/validation split (not random, to avoid leaking future information into training), then model training across four model types, and finally a consolidated evaluation and comparison stage. There is a clear start (raw hourly counts), intermediate stages (engineered features, trained models), and a final outcome (a compared, benchmarked set of models with a recommended best performer).

5.1 High Level Diagram (if applicable)

Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM



5.2 Low Level Diagram (if applicable)

At a lower level: raw rows (DateTime, Junction, Vehicles) are grouped per junction → sorted chronologically → temporal features (Hour, DayOfWeek, Month, IsWeekend, IsHoliday) are derived from DateTime → Lag_1h, Lag_24h, RollMean_3h and RollMean_24h are computed per junction (shifted so no future value leaks into a feature) → Junction is encoded (one-hot for Linear Regression, integer for Random Forest) → an 85/15 chronological split produces train/validation sets → each model is fit on train and scored on validation using MAE and RMSE.

5.3 Interfaces (if applicable)

The pipeline is script-based (not a live system), organized as a sequence of Python scripts (01_eda.py through 07_final_comparison.py) that pass data between stages via CSV/JSON files in a shared /data and /reports folder — each script reads the previous stage's output and writes its own, so the full pipeline can be re-run end-to-end or from any intermediate stage.

6 Performance Test

This section is where the project moves from “I built some models” to “here is proof of how well they actually work,” which is what separates this from a purely academic exercise.

The main constraints identified for this project were: forecast accuracy (MAE/RMSE) per model, fairness of comparison (avoiding data leakage), training time (especially for SARIMA and LSTM), and robustness across junctions with very different data volumes (Junction 4 has ~30% of the rows of Junctions 1-3).

These constraints were addressed by: using a strict chronological (not random) train/validation split for every model; keeping SARIMA and LSTM training scoped (trailing 90-day window for SARIMA, a modest 2-layer/32-16-unit architecture with early stopping for LSTM) to keep training time practical; and explicitly testing per-junction models to check robustness on the lower-data junction rather than assuming one combined model was good enough everywhere.

Test results are summarized below (see 6.3 Performance Outcome) — Random Forest (default settings) was the strongest overall performer; SARIMA's long-horizon forecast error was much higher, which is a genuine limitation of evaluating it as a blind 75-day-ahead forecast rather than a flaw in the seasonal modeling approach itself (it still visibly captured the daily seasonal shape).

Constraints considered: prediction accuracy (MAE, RMSE), training time, and data volume per junction. Memory/power constraints were not a concern for this project, since it runs as an offline batch pipeline rather than an embedded or real-time system.

N/A for this project — all identified constraints (accuracy, training time, per-junction robustness) were tested directly, with results in the table below.

6.1 Test Plan/ Test Cases

Each of the four models (Linear Regression, Random Forest, SARIMA, LSTM) was trained on the same chronological training window and evaluated on the same held-out validation window (the trailing ~15% of the timeline) using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). A separate test additionally trained per-junction Random Forest models to check whether junction-specific specialization improves on one combined model.

6.2 Test Procedure

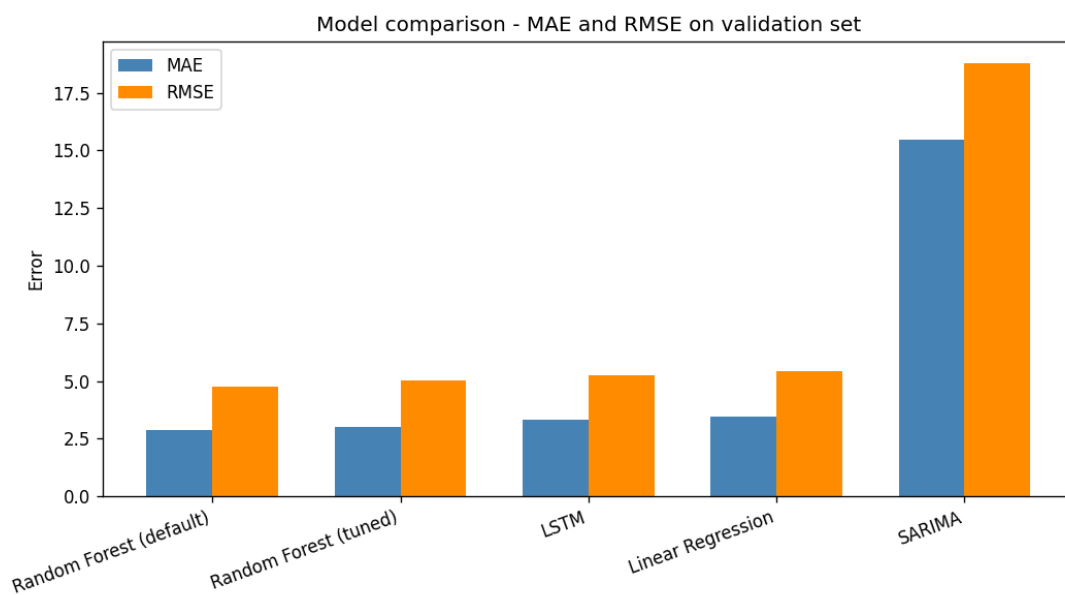
Data was split chronologically into train (Nov 2015 – mid Apr 2017) and validation (mid Apr 2017 – Jun 2017) sets before any model was fit, so no model ever saw validation-period data during training. Each model's predictions on the validation set were compared against actual vehicle counts to compute MAE and RMSE. For the per-junction test, results were further broken down by junction and by hour of day to see where errors concentrated.

6.3 Performance Outcome:

Random Forest (default settings) was the best all-around performer (MAE 2.89), narrowly ahead of the tuned version, the LSTM, and Linear Regression. SARIMA's error looks far higher because it was tested as a genuine 75-day-ahead blind forecast on Junction 1 only, not a like-for-like comparison with the feature-based models — it still visibly tracked the daily seasonal shape. A notable honest finding: grid-search tuning of the Random Forest did not beat the default configuration on this validation split, which is a real and worth-reporting outcome rather than a failed experiment.

Model	MAE	RMSE	Scope
Random Forest (default)	2.89	4.76	All junctions
Random Forest (tuned)	3.02	5.01	All junctions
LSTM (2-layer)	3.32	5.24	All junctions
Linear Regression	3.46	5.41	All junctions
SARIMA	15.49	18.79	Junction 1 only

Error analysis (Random Forest, tuned) showed MAE ranging from 1.92 at the quietest junction (Junction 4) up to 4.67 at the busiest (Junction 1), and errors roughly doubling during late-morning and evening rush hours compared to overnight hours. Training a separate Random Forest per junction improved Junction 1 (4.67 → 4.35 MAE) but was a wash or slightly worse for the lower-traffic junctions — a genuine trade-off between specialization and available training data per model, not a clean win.



7 My learnings

This internship gave me hands-on experience with the full applied-ML lifecycle, not just model training in isolation: data quality auditing, feature engineering specific to time-series data (lag/rolling features), and honest model evaluation. The most valuable lesson was technical — catching a data-leakage bug from an initial random train/validation split, and seeing directly how much that inflated my early results. I also learned that more complexity (LSTM, hyperparameter tuning) doesn't automatically mean better results; on this dataset, a well-configured Random Forest with good features outperformed both a tuned version of itself and a deep learning model. This kind of evidence-based comparison, rather than assuming the fanciest approach wins, is something I'll carry into future ML work and into placement interviews.

8 Future work scope

- Bring in external features such as weather data, which could explain some of the traffic spikes/dips currently only flagged as statistical outliers.
- Try an ensemble that blends Random Forest and SARIMA, to combine RF's overall accuracy with SARIMA's explicit seasonal shape.
- Extend the per-junction experiment into a proper hybrid — a dedicated model only for the busiest junction, combined model for the rest — based on this project's finding that splitting helps some junctions and not others.
- Deploy the best-performing model behind a simple API/dashboard so predicted vs. actual traffic could be viewed live rather than only in offline notebooks.
- Revisit LSTM with a larger sequence length or more training data once more historical records are available, since this project's version was constrained by both.

You can put some ideas that you could not work due to time limitation but can be taken in future.

